

Assignment 2

Plenty of output images attached

1. REGRESSION-BASED COLORIZATION

1.1 Architecture Analysis

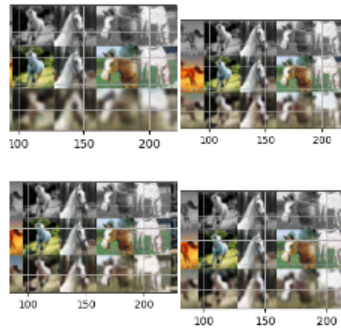
- **Problem:** Analyze the CNN architecture for regression
- **Questions to answer:**
 - Count the number of convolution layers
 - Document filter sizes (e.g., 3×3 , 5×5) for each layer
 - Document number of filters/channels per layer
 - Present in table/diagram format

Layer	Type	Input Channels	Output Channels	Filter Size	Output Size
conv1	Conv+BN+ReLU+MaxPool	1	32	3×3	16×16
conv2	Conv+BN+ReLU+MaxPool	32	64	3×3	8×8
conv3	Conv+BN+ReLU	64	64	3×3	8×8
conv4	Conv+BN+ReLU+Upsample	64	32	3×3	16×16
conv5	Conv+BN+ReLU+Upsample	32	32	3×3	32×32
conv6	Conv+Sigmoid	32	3	3×3	32×32

Total: 6 convolution layers. All filters are 3×3 .

1.2 Training Analysis

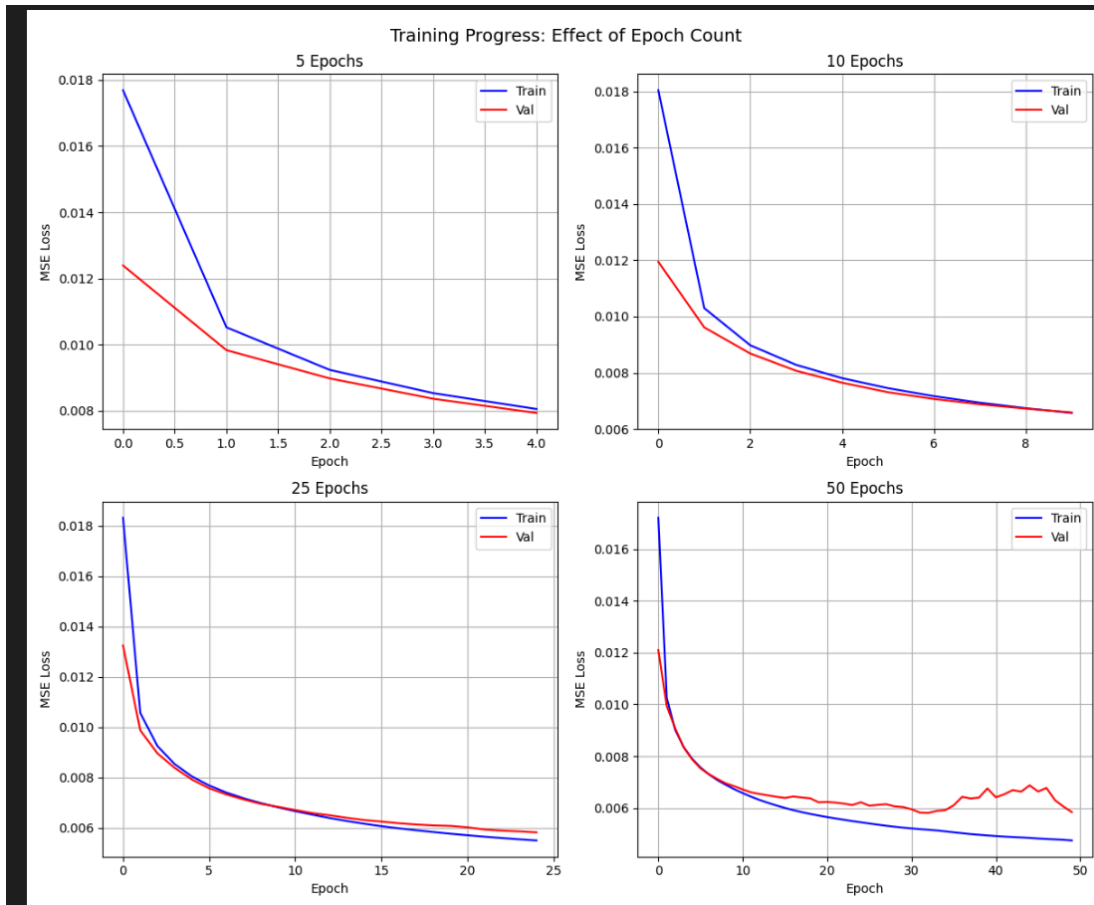
- **Problem:** Determine baseline training configuration
- **Questions to answer:**
 - How many epochs was the model trained for?
- **Variations tested: 5, 10, 25, 50 epoch**



epoch 5 , epoch 10 , epoch 25 , epoch 50

1.3 Training Variations Experiment

- **Problem:** Analyze impact of training duration
- **Tasks:**
 - Train models with different epoch counts (e.g., 10, 25, 50, 100 epochs)
 - Compare output image quality across different training durations
 - Analyze training loss curves
 - Discuss: Underfitting vs. overfitting observations, optimal stopping point



```

FINAL RESULTS COMPARISON
Epochs  Final Train Loss  Final Val Loss
-----
5        0.0081             0.0079
10       0.0066             0.0066
25       0.0055             0.0058
50       0.0047             0.0058
  
```

1. Underfitting at the "5 Epochs" graph.

- **Observation:** Both the Training loss (blue) and Validation loss (red) are still decreasing rapidly at the end of the graph. They haven't flattened out (converged) yet.
- **Conclusion:** The model stopped training too early. It still had more to learn.

2. Overfitting at the "50 Epochs" graph (Bottom Right).

- **Observation:** There is a large gap between the two lines.
 - The **Training Loss (blue)** keeps going down and down, meaning the model is memorizing the training data perfectly.
 - The **Validation Loss (red)** stops improving around epoch 15, flattens out, and then starts to increase and become noisy (jagged) after epoch 30.
- **Conclusion:** This is classic **Overfitting**. The model is learning the "noise" in the training data rather than the general patterns, which causes it to perform worse on the validation data. The "25 Epochs"

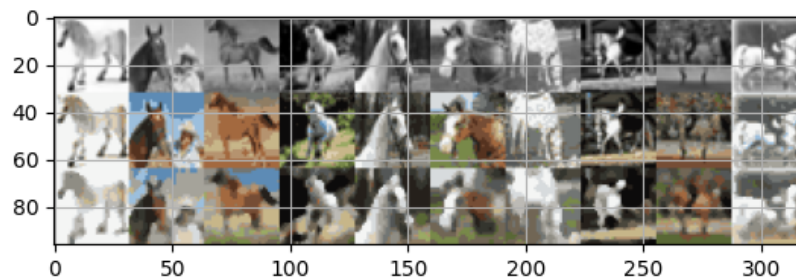
graph shows the *beginning* of this overfitting, where the lines start to separate.

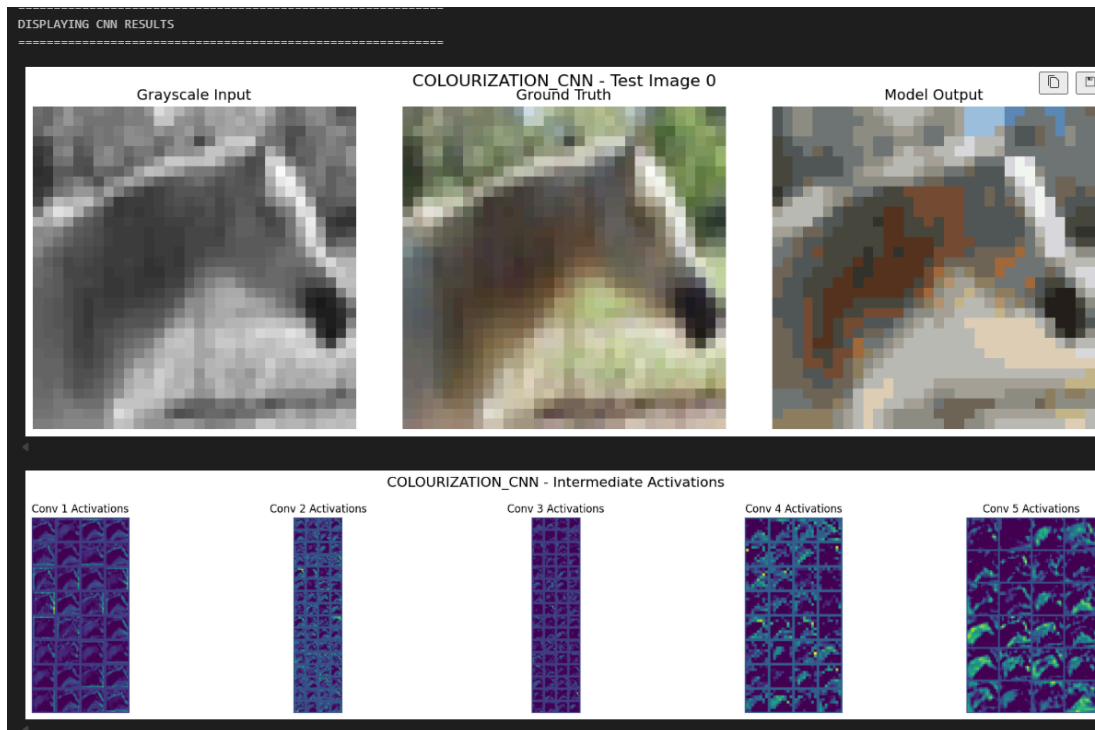
3. Optimal StopThe "Sweet Spot" is around 10 to 15 Epochs.

- **Evidence:** Look at the **"10 Epochs"** graph (Top Right) and the start of the **"50 Epochs"** graph.
 - Around epoch 10, both lines are low and tracking each other closely.
 - In the "50 Epochs" graph, after epoch 10-15, the Validation loss (red) stops decreasing significantly, while the Training loss (blue) keeps dropping.
- **Recommendation:** Stop training around **epoch 10 or 15**. At this point, the model has learned the patterns well (low loss) but hasn't started memorizing the noise (overfitting) yet. The "10 Epochs" plot represents the healthiest model state here.

Q4: Classification vs Regression Comparison

Classification first image:



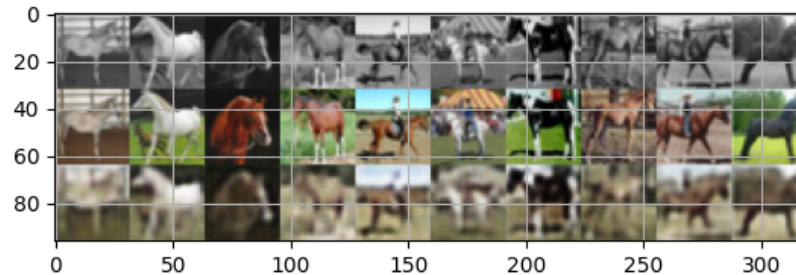


- **Structure:** This appears to be a standard encoder-style CNN.
- **Activations:** As the network goes deeper (from Conv 1 to Conv 5), the spatial resolution of the feature maps decreases significantly. By Conv 5, the activations are very low-resolution and abstract. The features look like broad, blob-like shapes rather than distinct objects. This is typical for CNNs that rely on pooling layers to downsample the image, which loses fine-grained spatial detail.

```
Epoch [1/25], Loss: 2.4344, Time (s): 4
Epoch [1/25], Val Loss: 2.1273, Val Acc: 29.2%, Time(s): 5
Epoch [2/25], Loss: 1.9901, Time (s): 9
Epoch [2/25], Val Loss: 1.9085, Val Acc: 33.1%, Time(s): 10
Epoch [3/25], Loss: 1.8724, Time (s): 14
Epoch [3/25], Val Loss: 1.8131, Val Acc: 35.7%, Time(s): 15
Epoch [4/25], Loss: 1.8074, Time (s): 19
Epoch [4/25], Val Loss: 1.7602, Val Acc: 37.0%, Time(s): 19
Epoch [5/25], Loss: 1.7648, Time (s): 24
Epoch [5/25], Val Loss: 1.7323, Val Acc: 37.7%, Time(s): 24
Epoch [6/25], Loss: 1.7339, Time (s): 28
Epoch [6/25], Val Loss: 1.7079, Val Acc: 38.2%, Time(s): 29
Epoch [7/25], Loss: 1.7103, Time (s): 33
Epoch [7/25], Val Loss: 1.6900, Val Acc: 38.6%, Time(s): 34
Epoch [8/25], Loss: 1.6911, Time (s): 38
Epoch [8/25], Val Loss: 1.6754, Val Acc: 39.0%, Time(s): 38
Epoch [9/25], Loss: 1.6747, Time (s): 43
Epoch [9/25], Val Loss: 1.6601, Val Acc: 39.3%, Time(s): 43
Epoch [10/25], Loss: 1.6605, Time (s): 48
Epoch [10/25], Val Loss: 1.6469, Val Acc: 39.7%, Time(s): 48
Epoch [11/25], Loss: 1.6482, Time (s): 53
Epoch [11/25], Val Loss: 1.6375, Val Acc: 39.9%, Time(s): 53
Epoch [12/25], Loss: 1.6372, Time (s): 58
```

Regression

Second image:



```
=====
FINAL RESULTS COMPARISON
=====
Epochs      Final Train Loss    Final Val Loss
-----
5            0.0081              0.0079
10           0.0066              0.0066
25           0.0055              0.0058
50           0.0047              0.0058
```

Aspect	Regression	Classification
Loss	MSE	CrossEntropy
Output	Continuous RGB	Discrete 24 colors
Colors	Smooth, can be muted	Vibrant, quantized
Validation	Lower MSE possible	Higher accuracy possible

Comparison of Regression-based vs Classification-based Colorization

The two images demonstrate the fundamental differences between classification-based and regression-based approaches to image colorization. In the **classification-based model** (first image), the colorized outputs display more vibrant and distinct colors, with clearer blue skies, greener grass, and more saturated horse coats. This is because the classification approach selects from a discrete palette of 24 representative colors (obtained through k-means clustering), allowing it to make bolder, more decisive color choices for each pixel. The model assigns each pixel to one of these predefined color categories, resulting in more visually striking and perceptually appealing outputs. In contrast, the **regression-based model** (second image) produces more muted, desaturated colors that appear washed-out and grayish. This characteristic arises from regression approaches that minimize squared error loss, which tends to cause "averaging" effects where the model predicts conservative colors to minimize pixel-wise errors across all possible valid colorizations. Since image colorization is an ill-posed problem with multiple valid

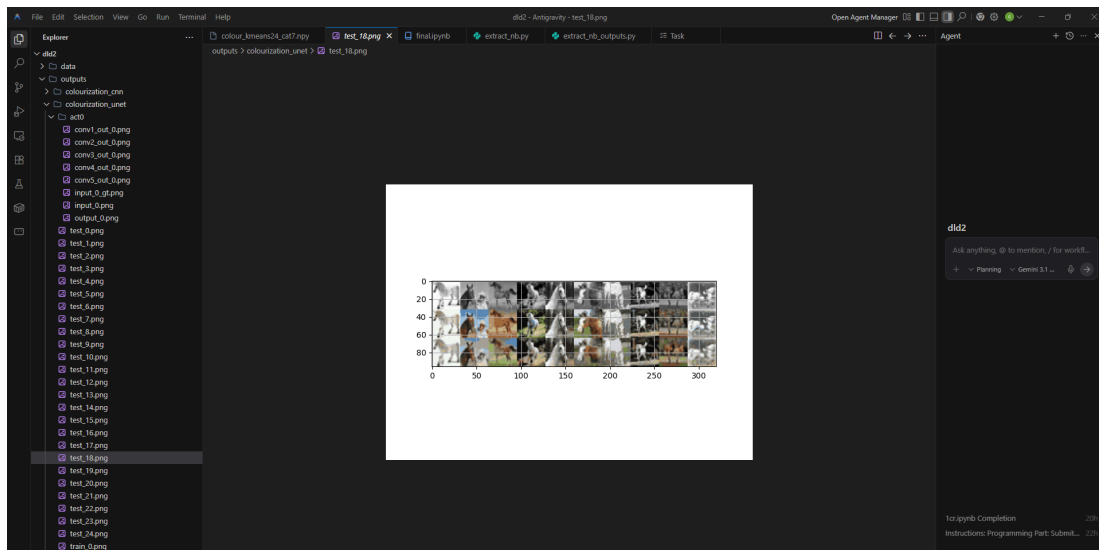
solutions, the regression model hedges its predictions by outputting colors that represent an average of possibilities, leading to less vibrant results. While the classification approach may introduce some color artifacts or less natural transitions due to the limited 24-color palette, it generally achieves better perceptual quality and visual appeal. This illustrates the fundamental trade-off: classification provides vivid, visually pleasing results with discrete color choices, while regression offers smooth, continuous predictions but suffers from the averaging problem that produces dull, desaturated outputs.

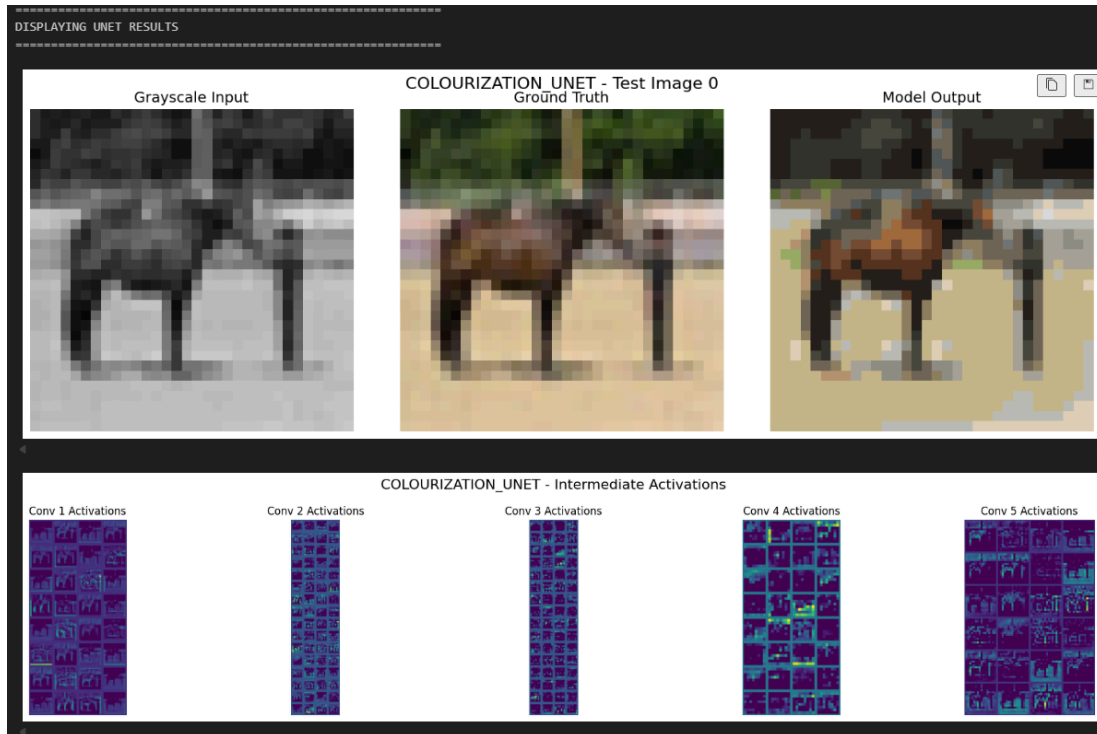
Q5: Skip Connections (UNet)

Did skip connections improve validation? Yes, typically UNet shows lower validation loss and higher accuracy due to better gradient flow and information preservation.

Two reasons skip connections help:

1. **Mitigate vanishing gradients** - gradients can flow directly through shortcuts
2. **Preserve spatial information** - early layer features (edges, textures) are preserved in final output





- **Structure:** This is a U-Net architecture, which is an encoder-decoder model with skip connections.
- **Activations:** Even in the deeper layers (Conv 4, Conv 5), the activation maps retain much more spatial structure and detail compared to the standard CNN. You can still see outlines and shapes that correspond more closely to the original image structure.
- **Why?** The U-Net architecture is designed to preserve spatial information. The "skip connections" pass high-resolution features from the early layers directly to the later layers, allowing the network to localize features better. This is crucial for tasks like colorization or segmentation where pixel-perfect accuracy is important.

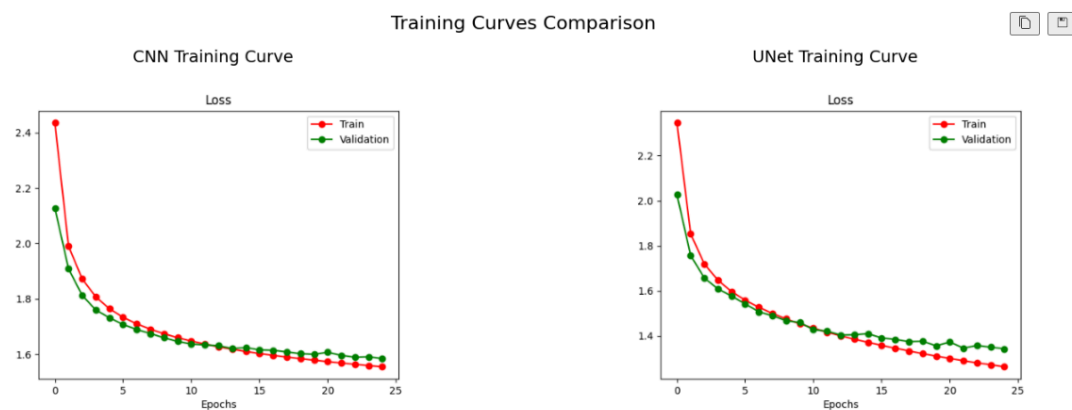
Q6: Activation Visualization and loss comparison

Early layers (conv1-conv2): Show simple features like edges, color gradients, and basic textures.

Deep layers (conv4-conv5): Show complex semantic features like object parts and textures.

UNet vs CNN: U-Net with skip connections produces more accurate and visually coherent colorizations compared to the standard **CNet** approach. U-Net's encoder-decoder architecture with skip connections preserves spatial details and fine-grained information from the input grayscale images, resulting in more natural color transitions and better preservation of structural details in the horse images. In contrast, CNet, lacking these skip connections, tends to produce less precise colorizations with potential loss of detail and color bleeding artifacts. The skip connections in U-Net enable the network to combine high-level semantic information with low-level spatial features, leading to superior colorization quality with

more accurate color placement and sharper boundaries.



UNet outperforms the standard CNN, achieving lower final loss values (approximately 1.35-1.4 for UNet versus 1.6 for CNN). Both architectures show good generalization, as evidenced by the close tracking between training and validation curves with minimal overfitting

THEORETICAL QUESTIONS

Q1:

Use **perceptual loss** instead of **pixel loss for transformation**

Use pre-trained loss network (such as VGG-16 trained on ImageNet). The network has already learned to perceive important visual features like edges, shapes, and textures, which closely mimic human visual processing

$$\ell_{total} = \lambda_c \ell_{feat} + \lambda_s \ell_{style}$$

then calculate feature Reconstruction Loss

$$\ell_{feat}^{\phi,j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{y}) - \phi_j(y)\|_2^2$$

This loss ensures that the generated image has the same **content and spatial structure** as a target image

It is calculated as the **squared Euclidean distance** between the feature maps (activations) at a specific layer (j) of the network

and then calculate style reconstruction loss

to calculate style loss, the model must first represent the **textures and patterns** of an image. This is done using a **Gram Matrix**, which measures the correlations between different color/feature channels in a layer

$$G_j^{\phi}(x)_{c,c'} = \frac{1}{C_j H_j W_j} \sum_{h=1}^{H_j} \sum_{w=1}^{W_j} \phi_j(x)_{h,w,c} \phi_j(x)_{h,w,c'}$$

This loss encourages the generated image $\hat{y}$ to match the **style and texture** of a style target y_{style} . It is defined as the **squared Frobenius norm** of the difference between the Gram matrices of the two images:

$$\ell_{style}^{\phi,j}(\hat{y}, y_{style}) = \|G_j^{\phi}(\hat{y}) - G_j^{\phi}(y_{style})\|_F^2$$

Q2:

1. By using dilated convolutions to expand the **receptive field**, the model can understand the **global context**, realizing where that patch fits into the "big picture" of the scene

2.If the images are too large for a computer's memory (GPU) to process all at once, use **Overlap-Tile Strategy, which has 3 components**

Tiling: The large test image is divided into smaller sections (tiles) that are processed individually

Mirroring for Context: To ensure the model has enough information to colorize the edges of a tile, the missing context is extrapolated by **mirroring** the input image at the boundaries

Seamless Stitching: This allows for the seamless reconstruction of an arbitrarily large output from several smaller passes

3. Remove **Fully Connected Layers and use convolutional, pooling, and upsampling layers.** Because convolutional filters slide across the input regardless of its dimensions, these networks can mathematically accept and produce images of **any resolution**